EDGE INTELLIGENCE

LAB - 4

25MML0039

VISHWA M

TASK 1: ARTIFICIAL NEURAL NETWORK USING DL

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Replace your manual np.load lines with this:
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

```
print("Before normalization :", x_train.shape, y_train.shape)

model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Train
model.fit(x_train, y_train, epochs=2)
```

```
# Evaluate
loss, acc = model.evaluate(x_test, y_test)
print(f"\nAccuracy = {acc}")
```

```
# Predict first 5 images
pred = model.predict(x_test[:5])
print("\nPredictions:\n", pred)

model.summary()
```

```
import matplotlib.pyplot as plt
plt.imshow(x_test[0],cmap='gray')
plt.title("predicted:" +str(pred))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Before normalization : (60000, 28, 28) (60000,)
Epoch 1/2
/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `inp
ut_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.
  super().__init__(**kwargs)
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.2151 - loss: 117.9844
Epoch 2/2
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.2913 - loss: 1.9200
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.2145 - loss: 2.0355

Accuracy = 0.20800000429153442
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step


Predictions:
 [[1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
  1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
  1.12041324e-01 1.14244498e-01]
 [1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
  1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
  1.12041324e-01 1.14244498e-01]
 [0.00000000e+00 1.00000000e+00 1.25715510e-30 0.00000000e+00
  0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
  0.00000000e+00 0.00000000e+00]
 [1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
  1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
  1.12041324e-01 1.14244498e-01]
 [1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
  1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
  1.12041324e-01 1.14244498e-01]]
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 128)	100,480
dense_3 (Dense)	(None, 10)	1,290

Total params: 101,772 (397.55 KB)

Trainable params: 101,770 (397.54 KB)

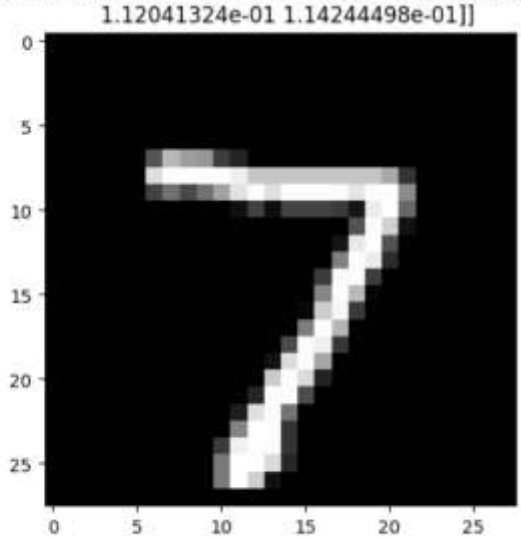
Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

[16]:
```
Text(0.5, 1.0, 'predicted:[[1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01\n  1.13865547e-01 1.048567
97e-01 9.96315703e-02 8.37114528e-02\n  1.12041324e-01 1.14244498e-01]\n [1.09122708e-01 4.27314229e-02 1.08976804
e-01 1.10817797e-01\n  1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02\n  1.12041324e-01 1.14244498e-0
1]\n [0.00000000e+00 1.00000000e+00 1.25715510e-30 0.00000000e+00\n  0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00\n  0.00000000e+00 0.00000000e+00]\n [1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01\n
1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02\n  1.12041324e-01 1.14244498e-01]\n [1.09122708e-01 4.
27314229e-02 1.08976804e-01 1.10817797e-01\n  1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02\n  1.120
41324e-01 1.14244498e-01]]')
```

predicted:[[1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
1.12041324e-01 1.14244498e-01]
[1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
1.12041324e-01 1.14244498e-01]
[0.00000000e+00 1.00000000e+00 1.25715510e-30 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00]
[1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
1.12041324e-01 1.14244498e-01]
[1.09122708e-01 4.27314229e-02 1.08976804e-01 1.10817797e-01
1.13865547e-01 1.04856797e-01 9.96315703e-02 8.37114528e-02
1.12041324e-01 1.14244498e-01]]

1.12041324e-01 1.14244498e-01]]



[17]:
```python
import pickle

# Save the model
with open('mnist_model.pkl', 'wb') as f:
    pickle.dump(model, f)

# Load the model
with open('mnist_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)

# Verify
loaded_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 128)	100,480
dense_3 (Dense)	(None, 10)	1,290

Total params: 101,772 (397.55 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2 (12.00 B)

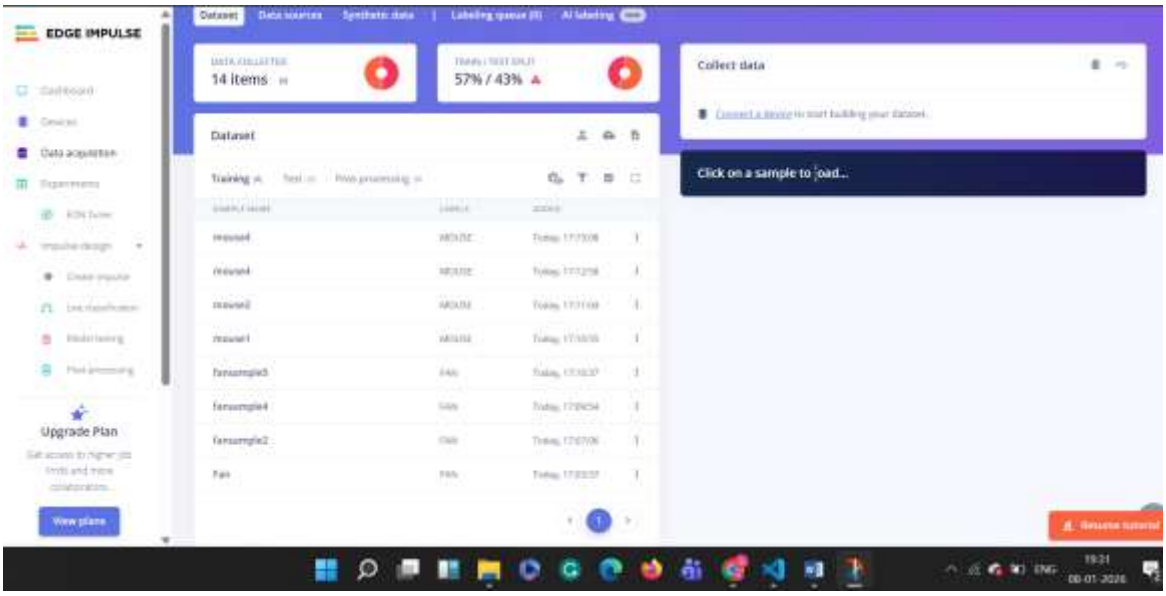
TASK 2: EDGE IMPLUSE

STEPS:

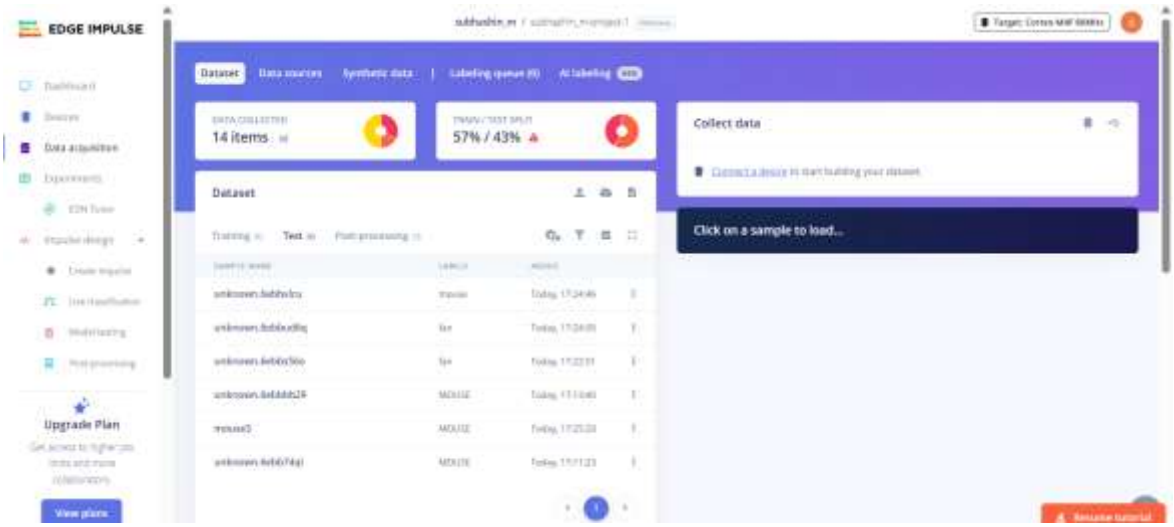
1. Create an account on **Edge Impulse**.
2. Login to the Edge Impulse platform.

3. Create a **new project**.
4. Click **Data Acquisition**.
5. Select **Connect to device**.
6. Choose **Use mobile phone** and scan the QR code.
7. Connect the mobile phone as a camera.
8. Capture sample images.
9. Collect images for **training** and **testing**.
10. Rename images with meaningful names.
11. Label all images correctly.

Training



Testing


EDGE IMPULSE
Dataset
Data sources
Synthetic data
Data acquisition
Upgrade Plan
View plans
14 items
57% / 43%
Collect data
Dataset
Test
Click on a sample to load...